

✓ Install Required Libraries

```
!pip install fairlearn shap lime ucimlrepo seaborn matplotlib scikit-learn pandas
```

```
Collecting fairlearn
```

```
  Downloading fairlearn-0.14.0-py3-none-any.whl.metadata (7.3 kB)
```

```
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.43.0)
```

```
Collecting lime
```

```
  Downloading lime-0.2.0.1.tar.gz (275 kB)
```

```
275.7/275.7 kB 1.5 MB/s eta 0:00:00
```

```
  Preparing metadata (setup.py) ... done
```

```
Collecting ucimlrepo
```

```
  Downloading ucimlrepo-0.0.7-py3-none-any.whl.metadata (5.5 kB)
```

```
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.8.0)
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.4.2)
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
```

```
Requirement already satisfied: narwhals>=1.14.0 in /usr/local/lib/python3.12/dist-packages (1.17.0)
```

```
Requirement already satisfied: numpy>=1.24.4 in /usr/local/lib/python3.12/dist-packages (2.0.2)
```

```
Requirement already satisfied: scipy>=1.9.3 in /usr/local/lib/python3.12/dist-packages (1.12.0)
```

```
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (4.67.1)
```

```
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (24.1)
```

```
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (0.0.8)
```

```
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-packages (0.60.0)
```

```
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/dist-packages (0.43.0)
```

```
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (3.0.0)
```

```
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (0.22.0)
```

```
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/python3.12/dist-packages (2024.7.4)
```

```
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (1.2.1)
```

```
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packages (0.10.0)
```

```
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (4.54.1)
```

```
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (1.4.5)
```

```
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (10.4.0)
```

```
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (3.1.4)
```

```
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (2.9.0)
```

```
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (1.4.2)
```

```
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (3.5.0)
```

```
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (2024.2)
```

```
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (2024.7)
```

```
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (1.17.0)
```

```
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (3.3)
```

```
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (2.36.0)
```

```
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (2024.12.1)
```

```
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (0.4.0)
```

```
Downloading fairlearn-0.14.0-py3-none-any.whl (135 kB)
```

```
135.5/135.5 kB 8.5 MB/s eta 0:00:00
```

```
Downloading ucimlrepo-0.0.7-py3-none-any.whl (8.0 kB)
```

```
Building wheels for collected packages: lime
```

```
  Building wheel for lime (setup.py) ... done
```

```
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834 sha256=
```

```
  Stored in directory: /root/.cache/pip/wheels/e7/5d/0e/4b4fff9a47468fed5633211f
```

```
Successfully built lime
Installing collected packages: ucimlrepo, lime, fairlearn
Successfully installed fairlearn-0.14.0 lime-0.2.0.1 ucimlrepo-0.0.7
```

✓ Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from ucimlrepo import fetch_ucirepo

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

from fairlearn.metrics import (
    MetricFrame,
    selection_rate,
    false_positive_rate,
    true_positive_rate
)

import shap
import lime
import lime.lime_tabular

import warnings
warnings.filterwarnings("ignore")
```

✓ Introduction

```
print("""
Ethical AI Analysis and Explainability

Objectives:
1. Train a machine learning model.
2. Evaluate model performance.
3. Analyze fairness across demographic groups.
```

```
4. Explain model decisions using SHAP and LIME.
"""
```

Ethical AI Analysis and Explainability

Objectives:

1. Train a machine learning model.
2. Evaluate model performance.
3. Analyze fairness across demographic groups.
4. Explain model decisions using SHAP and LIME.

✓ Load Dataset

```
adult = fetch_ucirepo(id=2)

X = adult.data.features
y = adult.data.targets

df = pd.concat([X, y], axis=1)

print(df.head())
print(df.shape)
```

```
   age  workclass  fnlwgt  education  education-num  \
0   39   State-gov   77516   Bachelors             13
1   50  Self-emp-not-inc   83311   Bachelors             13
2   38     Private  215646   HS-grad              9
3   53     Private  234721     11th              7
4   28     Private  338409   Bachelors             13

   marital-status  occupation  relationship  race  sex  \
0  Never-married  Adm-clerical  Not-in-family  White  Male
1  Married-civ-spouse  Exec-managerial    Husband  White  Male
2        Divorced  Handlers-cleaners  Not-in-family  White  Male
3  Married-civ-spouse  Handlers-cleaners    Husband  Black  Male
4  Married-civ-spouse  Prof-specialty      Wife  Black  Female

   capital-gain  capital-loss  hours-per-week  native-country  income
0         2174           0           40  United-States  <=50K
1           0           0           13  United-States  <=50K
2           0           0           40  United-States  <=50K
3           0           0           40  United-States  <=50K
4           0           0           40         Cuba  <=50K
(48842, 15)
```

✓ Explore Dataset

```
print(df.info())
```

```
print("\nMissing Values:")  
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 48842 entries, 0 to 48841  
Data columns (total 15 columns):  
#   Column                Non-Null Count  Dtype  
---  ---  
0   age                   48842 non-null  int64  
1   workclass             47879 non-null  object  
2   fnlwgt                48842 non-null  int64  
3   education             48842 non-null  object  
4   education-num         48842 non-null  int64  
5   marital-status        48842 non-null  object  
6   occupation            47876 non-null  object  
7   relationship          48842 non-null  object  
8   race                  48842 non-null  object  
9   sex                   48842 non-null  object  
10  capital-gain           48842 non-null  int64  
11  capital-loss           48842 non-null  int64  
12  hours-per-week        48842 non-null  int64  
13  native-country        48568 non-null  object  
14  income                48842 non-null  object  
dtypes: int64(6), object(9)  
memory usage: 5.6+ MB  
None
```

Missing Values:

age	0
workclass	963
fnlwgt	0
education	0
education-num	0
marital-status	0
occupation	966
relationship	0
race	0
sex	0
capital-gain	0
capital-loss	0
hours-per-week	0
native-country	274
income	0

dtype: int64

Dataset Description

```
print("""  
Dataset: Adult Census Income
```

Goal:

Predict whether a person's income exceeds \$50,000 annually.

Sensitive Attribute:
sex

Target Variable:
income
""")

Dataset: Adult Census Income

Goal:
Predict whether a person's income exceeds \$50,000 annually.

Sensitive Attribute:
sex

Target Variable:
income

✓ Data Preprocessing

Remove Missing Values

```
df = df.replace("?", np.nan)

df.dropna(inplace=True)

print("Dataset Shape After Cleaning:")
print(df.shape)
```

```
Dataset Shape After Cleaning:
(45222, 15)
```

Define Target Variable

```
df["income"] = df["income"].apply(
    lambda x: 1 if ">50K" in str(x) else 0
)
```

Sensitive Feature

```
sensitive_feature = df["sex"]
```

```
print(df["sex"].value_counts())
```

```
sex
Male      30527
Female    14695
Name: count, dtype: int64
```

Feature Selection

```
X = df.drop("income", axis=1)
```

```
y = df["income"]
```

✓ Encode Categorical Variables

```
X = pd.get_dummies(
    X,
    drop_first=True
)

print(X.shape)
```

```
(45222, 96)
```

✓ Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

Sensitive Feature Split

```
sensitive_train = sensitive_feature.loc[X_train.index]
sensitive_test = sensitive_feature.loc[X_test.index]
```

✓ Train Logistic Regression Model

```
model = LogisticRegression(  
    max_iter=1000  
)  
  
model.fit(  
    X_train,  
    y_train  
)
```

▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=1000)

✓ Predictions

```
y_pred = model.predict(X_test)
```

✓ Model Evaluation

Accuracy

```
accuracy = accuracy_score(  
    y_test,  
    y_pred  
)  
  
print("Accuracy:", accuracy)
```

Accuracy: 0.8390270867882809

✓ Confusion Matrix

```
cm = confusion_matrix(  
    y_test,  
    y_pred  
)  
  
print(cm)
```

```
[[6311  492]  
 [ 964 1278]]
```

✓ Visualize Confusion Matrix

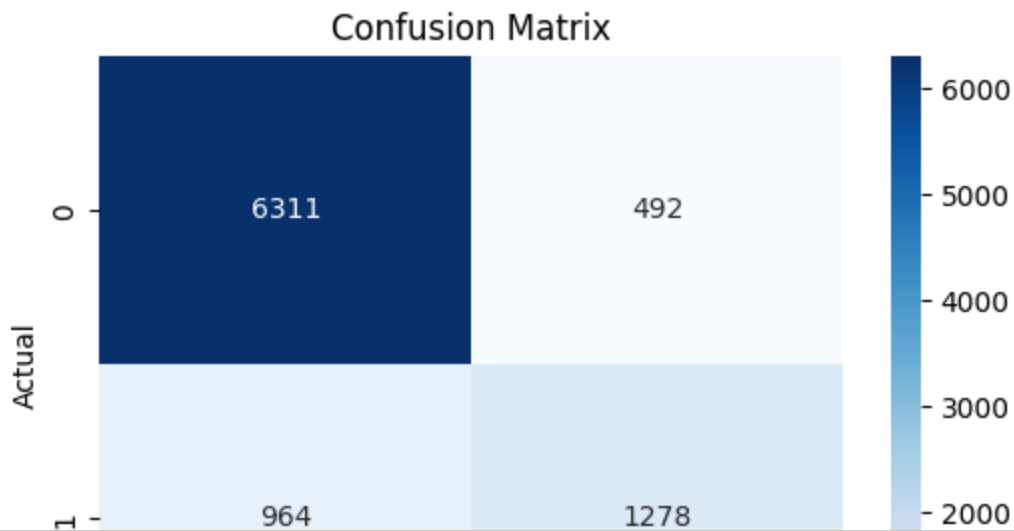
```
plt.figure(figsize=(6,4))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap="Blues"
)

plt.title("Confusion Matrix")

plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



✓ Classification Report

```
print(
    classification_report(
        y_test,
        y_pred
    )
)
```

	precision	recall	f1-score	support
0	0.87	0.93	0.90	6803

	1	0.72	0.57	0.64	2242
accuracy				0.84	9045
macro avg		0.79	0.75	0.77	9045
weighted avg		0.83	0.84	0.83	9045

Model Evaluation Discussion

```
print("""
Model Evaluation Discussion

Accuracy measures the percentage of
correct predictions.

Precision measures how many predicted
positive cases are actually positive.

Recall measures how many true positive
cases were identified.

F1-score balances precision and recall.

These metrics evaluate overall model
performance before analyzing fairness.
""")
```

```
Model Evaluation Discussion

Accuracy measures the percentage of
correct predictions.

Precision measures how many predicted
positive cases are actually positive.

Recall measures how many true positive
cases were identified.

F1-score balances precision and recall.

These metrics evaluate overall model
performance before analyzing fairness.
```

✓ Fairness Analysis

MetricFrame

```
metric_frame = MetricFrame(
    metrics={
        "selection_rate": selection_rate,
        "false_positive_rate": false_positi
        "true_positive_rate": true_positive
    },
    y_true=y_test,
    y_pred=y_pred,
    sensitive_features=sensitive_test
)
```

Display Fairness Metrics

```
print(metric_frame.by_group)
```

	selection_rate	false_positive_rate	true_positive_rate
sex			
Female	0.080094	0.028126	0.467422
Male	0.252599	0.100192	0.589201

```
print(metric_frame.overall)
```

selection_rate	0.195688
false_positive_rate	0.072321
true_positive_rate	0.570027
dtype: float64	

✓ Visualize Fairness Metrics

```
metric_frame.by_group.plot(
    kind="bar",
    figsize=(10,6)
)

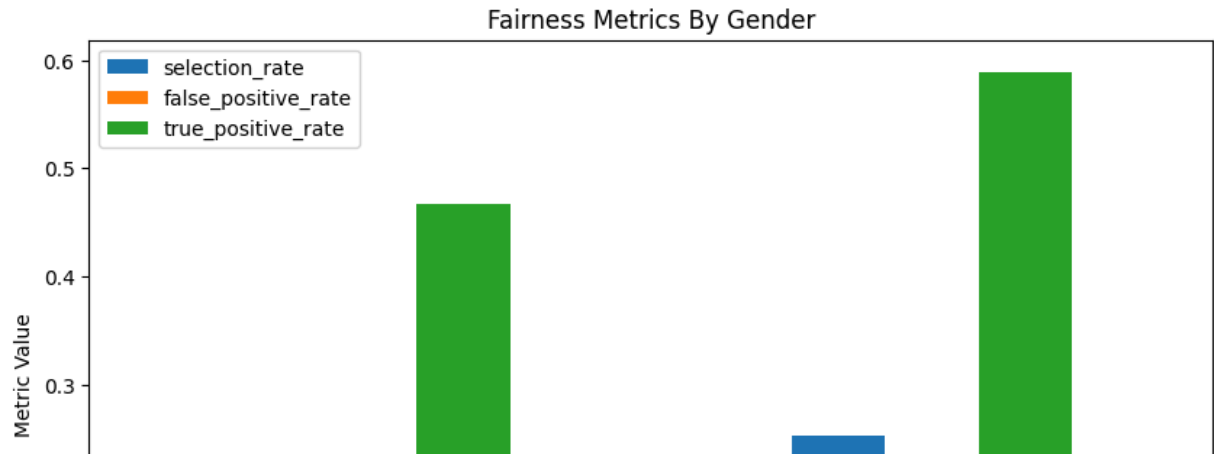
plt.title(
    "Fairness Metrics By Gender"
)

plt.ylabel(
    "Metric Value"
)

plt.xlabel(
    "Gender"
)

plt.xticks(rotation=0)
```

```
plt.show()
```



✓ Fairness Discussion

```
print("""
Fairness Interpretation:

Selection Rate:
Shows how frequently individuals
receive positive predictions.

False Positive Rate:
Measures incorrect positive
predictions across groups.

True Positive Rate:
Measures correctly identified
positive records across groups.

Differences may indicate
potential bias.
""")
```

```
Fairness Interpretation:

Selection Rate:
Shows how frequently individuals
receive positive predictions.

False Positive Rate:
Measures incorrect positive
predictions across groups.

True Positive Rate:
Measures correctly identified
```

positive records across groups.

Differences may indicate
potential bias.

✓ Fairness Interpretation

```
selection_gap = (  
    metric_frame.by_group["selection_rate"].max()  
    - metric_frame.by_group["selection_rate"].min()  
)  
  
print("Selection Rate Difference:", selection_gap)  
  
print("""  
Fairness Interpretation  
  
Selection Rate:  
Measures how frequently a group receives  
positive predictions.  
  
False Positive Rate:  
Measures how frequently positive outcomes  
are incorrectly assigned.  
  
True Positive Rate:  
Measures how frequently actual positive  
cases are correctly identified.  
  
Large differences between groups may  
indicate potential bias.  
""")
```

Selection Rate Difference: 0.17250474731206814

Fairness Interpretation

Selection Rate:
Measures how frequently a group receives
positive predictions.

False Positive Rate:
Measures how frequently positive outcomes
are incorrectly assigned.

True Positive Rate:
Measures how frequently actual positive
cases are correctly identified.

Large differences between groups may indicate potential bias.

✓ Global Explainability With SHAP

Create SHAP Explainer

```
explainer = shap.Explainer(  
    model,  
    X_train  
)
```

WARNING:shap:Background dataset has 36177 samples but max_samples=100. Subsampli

Generate SHAP Values

```
shap_values = explainer(  
    X_test  
)
```

SHAP Summary Plot

```
shap.plots.beeswarm(  
    shap_values,  
    max_display=15  
)
```

File ["/tmp/ipykernel_1193/4013950391.py"](#), line 5

```
^  
SyntaxError: invalid syntax
```

Next steps: [Explain error](#)

✓ SHAP Interpretation

```
print("""
SHAP Global Interpretation
```

The SHAP beeswarm plot identifies the features with the greatest influence on income predictions.

Features shown at the top contribute the most to model decisions.

The SHAP bar chart ranks overall feature importance across the dataset.

```
""")
```

SHAP Global Interpretation

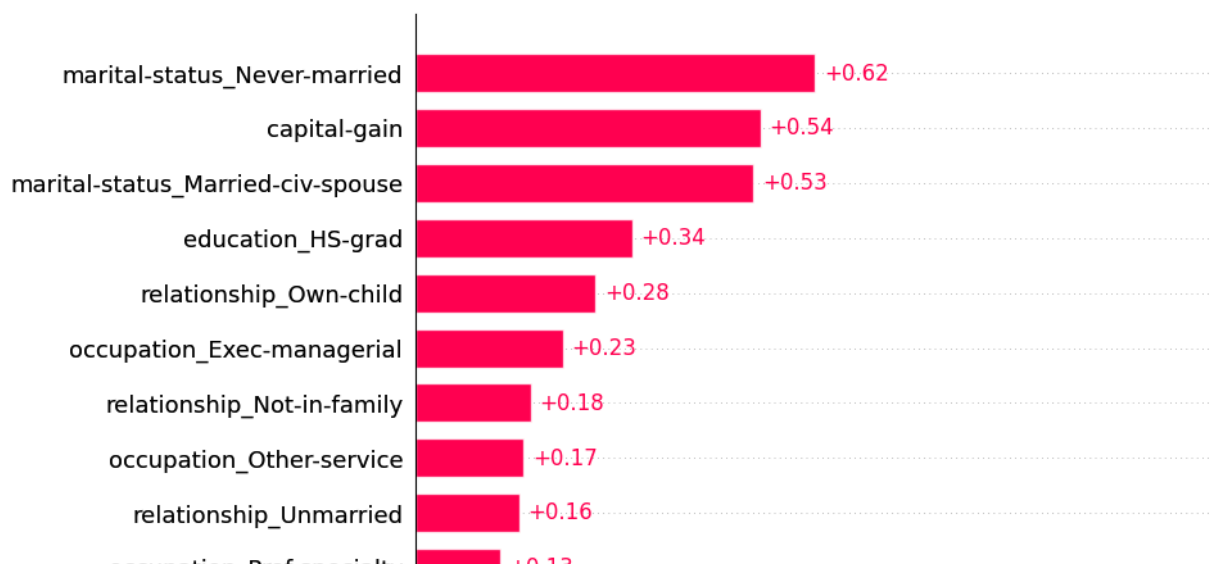
The SHAP beeswarm plot identifies the features with the greatest influence on income predictions.

Features shown at the top contribute the most to model decisions.

The SHAP bar chart ranks overall feature importance across the dataset.

✓ SHAP Bar Plot

```
shap.plots.bar(
    shap_values,
    max_display=15
)
```



✓ Local Explainability With SHAP

Waterfall Plot

◆ Gemini

```
shap.plots.waterfall(  
    shap_values[0]  
)  
`
```

File ["/tmp/ipykernel_1193/1243492300.py"](#), line 4

SyntaxError: invalid syntax

Next steps: [Explain error](#)

```
print("""  
SHAP Local Interpretation
```

The waterfall plot explains the prediction for one specific individual.

Positive SHAP values increase the probability of earning more than \$50K.

Negative SHAP values decrease the probability.

```
This provides transparency into the  
reasoning behind individual predictions.  
""")
```

```
SHAP Local Interpretation
```

The waterfall plot explains the prediction for one specific individual.

Positive SHAP values increase the probability of earning more than \$50K.

Negative SHAP values decrease the probability.

This provides transparency into the

reasoning behind individual predictions.

✓ LIME Explainability

Create Explainer

```
lime_explainer = lime.lime_tabular.LimeTabu
    training_data=np.array(X_train),
    feature_names=X_train.columns.tolist(),
    class_names=["<=50K", ">50K"],
    mode="classification"
)
```

Explain One Prediction

```
lime_exp = lime_explainer.explain_instance(
    X_test.iloc[0].values,
    model.predict_proba,
    num_features=10
)
```

✓ Display Explanation

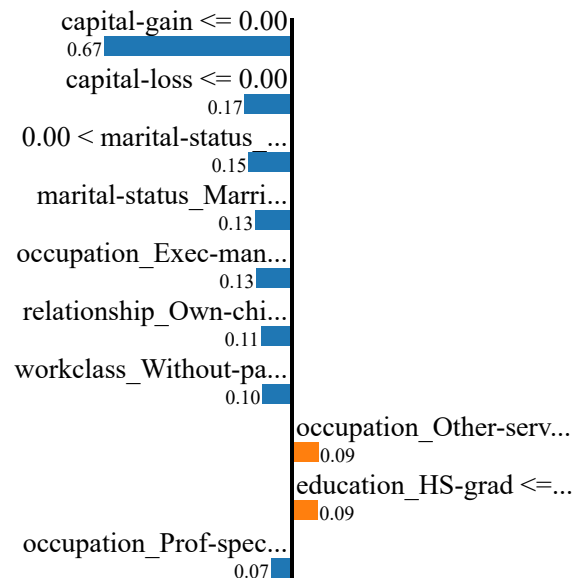
```
lime_exp.show_in_notebook(
    show_table=True
)
```


Prediction probabilities

Income <=50K 0.98
 Income >50K 0.02

Income <=50K

Income >50K



Feature

Value

capital-gain	0.00
capital-loss	0.00
marital-status_Never-married	1.00
marital-status_Married-civ-spouse	0.00
occupation_Exec-managerial	0.00
relationship_Own-child	1.00
workclass_Without-pay	0.00
occupation_Other-service	0.00
education_HS-grad	0.00
occupation_Prof-specialty	0.00

✓ Save LIME to HTML

```
lime_exp.save_to_file(
    "lime_explanation.html"
)
```

LIME Discussion

```
print("""
LIME Interpretation

LIME explains an individual prediction.
```

It identifies which features contributed positively and negatively to the model's decision.

LIME improves transparency and user trust by showing how predictions were made.

```
""")
```

✓ Ethical Analysis

```
print("""  
ETHICAL ANALYSIS
```

1. FAIRNESS

The model may produce different outcomes across demographic groups.

Differences in selection rates could indicate bias inherited from historical data.

2. TRANSPARENCY

SHAP and LIME provide insight into how the model reaches decisions.

3. ACCOUNTABILITY

Transparent models improve trust and support responsible AI deployment.

4. RECOMMENDATIONS

- Monitor fairness metrics.
 - Regularly audit AI systems.
 - Use Fairlearn mitigation tools.
 - Remove proxy attributes.
 - Retrain using balanced data.
- ```
""")
```

```
ETHICAL ANALYSIS
```

### 1. FAIRNESS

The model may produce different outcomes across demographic groups.

Differences in selection rates could indicate bias inherited from historical data.

## 2. TRANSPARENCY

SHAP and LIME provide insight into how the model reaches decisions.

## 3. ACCOUNTABILITY

Transparent models improve trust and support responsible AI deployment.

## 4. RECOMMENDATIONS

- Monitor fairness metrics.
- Regularly audit AI systems.
- Use Fairlearn mitigation tools.
- Remove proxy attributes.
- Retrain using balanced data.

## ✓ Save Model

```
joblib.dump(
 model,
 "adult_income_model.pkl"
)

print("Model Saved Successfully")
```

Model Saved Successfully

## ✓ Generate Results File

```
results = pd.DataFrame({
 "Actual": y_test,
 "Predicted": y_pred
})
```

```
results.to_csv(
 "predictions.csv",
 index=False
)

results.head()
```

|              | Actual | Predicted |  |
|--------------|--------|-----------|-----------------------------------------------------------------------------------|
| <b>39871</b> | 0      | 0         |                                                                                   |
| <b>19367</b> | 1      | 1         |                                                                                   |
| <b>3454</b>  | 0      | 0         |                                                                                   |
| <b>38198</b> | 1      | 0         |                                                                                   |
| <b>14455</b> | 0      | 0         |                                                                                   |

Next steps: [Generate code with results](#) [New interactive sheet](#)

## ✓ Conclusion

```
print("""
CONCLUSION
```

This project developed a logistic regression model using the Adult Income dataset.

The model was evaluated using:

- Accuracy
- Confusion Matrix
- Classification Report

Fairness was assessed using:

- Selection Rate
- False Positive Rate
- True Positive Rate

Explainability was performed using:

- SHAP Summary Plot
- SHAP Waterfall Plot
- LIME Explanations

Results demonstrate the importance

of combining model performance with fairness and transparency to build responsible AI systems.

```
"""\
```

## CONCLUSION

This project developed a logistic regression model using the Adult Income dataset.

The model was evaluated using:

- Accuracy
- Confusion Matrix
- Classification Report

Fairness was assessed using:

- Selection Rate
- False Positive Rate
- True Positive Rate

Explainability was performed using:

- SHAP Summary Plot
- SHAP Waterfall Plot
- LIME Explanations

Results demonstrate the importance of combining model performance with fairness and transparency to build responsible AI systems.